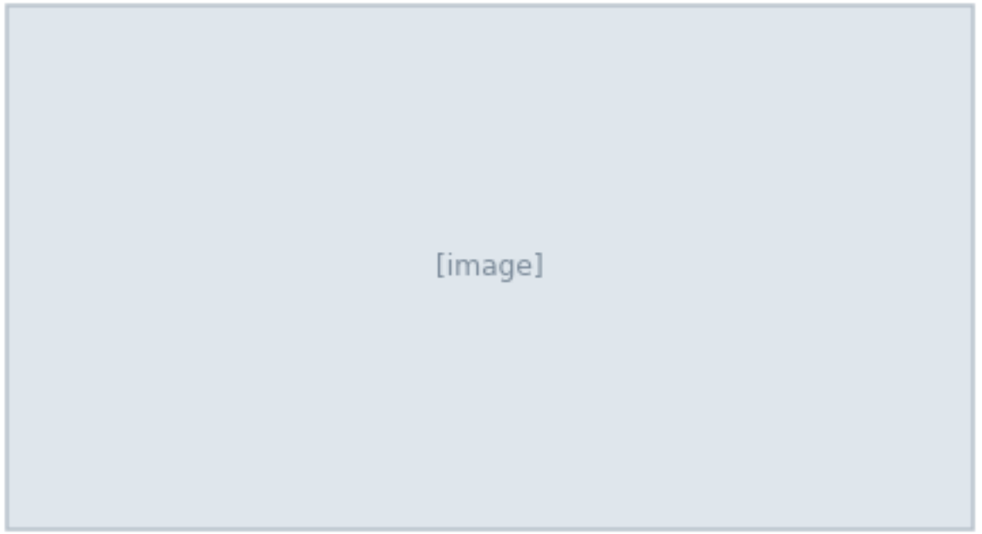
ARCHITECTURE BRIEF

Agent-Native Apps

Software designed for the user's own agent.

Cursor + Codex + shared context + tool access



Build for the user's agent, not just the user.

The thesis

The workflow changes when the agent owns the loop

Old model	New model	What changes
App embeds a chatbot. The model sees only local app context, so it can answer questions but cannot finish the job. Result: the user still does the hard part outside the app.	The user's agent owns the context and uses the app as one surface among many. Result: the app becomes a place where work actually gets done, with the user's broader memory attached.	Moat shifts to shared context, tool access, workflow design, and persistent memory. If those four pieces are weak, the model layer is just decoration.

If the app is not usable by the user's agent, it is only partially agent-native.

The architecture is a loop, not a router.

Model inversion

App with an agent vs. agent with an app

Old model: the app contains the agent. New model: the agent contains the app surface and brings its own context.

Five principles

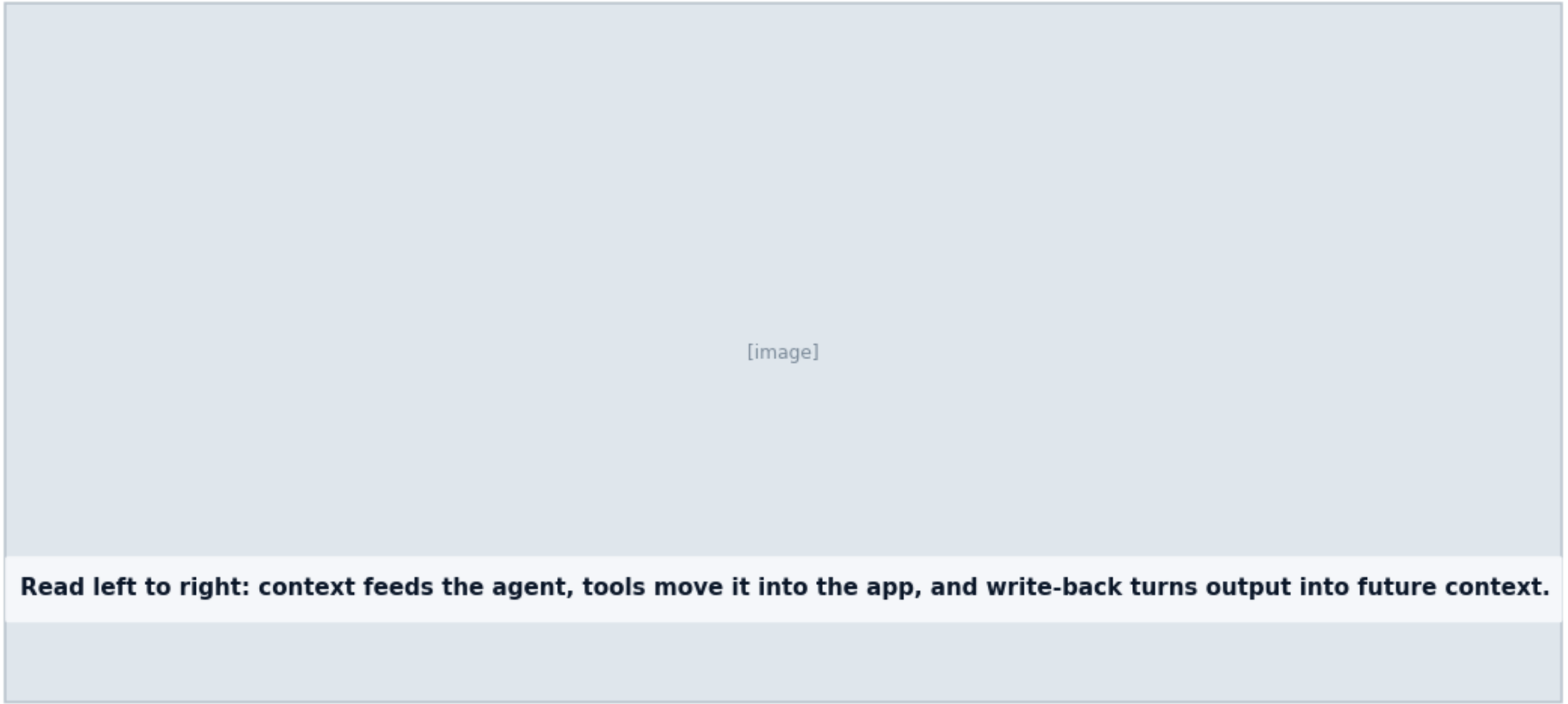
The checklist that separates real agent-native systems from AI-colored software

Parity	Granularity	Composability	Emergent capability	Improvement over time
Anything the user can do in UI, the agent can do through tools. If parity is missing, the agent is only a suggestion engine.	Tools are atomic primitives. Features come from prompt-defined outcomes. Keep the actions small so agents can combine them.	New capabilities come from new prompt + tool combinations. The product gets smarter when the same pieces work in new sequences.	The system should handle requests you did not pre-script. That is the difference between automation and a real agent surface.	Context and memory should make the system better with use. If it does not learn from the workspace, it will not compound value.

If the product cannot satisfy these five checks, it is still too constrained.

Reference architecture

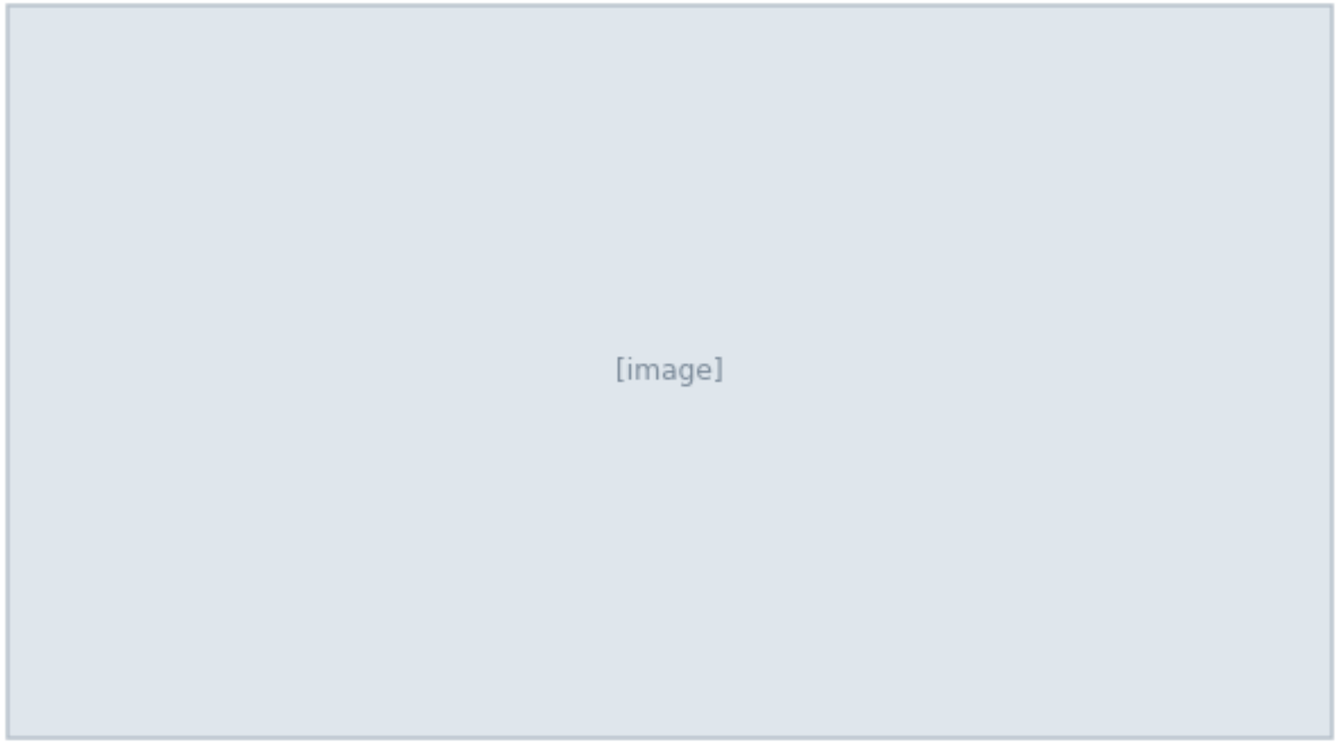
Context → agent loop → shared surface → outcome → memory



Read left to right: context feeds the agent, tools move it into the app, and write-back turns output into future context.

Tool layer

MCP, CLI, API, and browser access are the unlock



Parity

If a user can do it, the agent needs a path to do it too. Without that path, the feature is just a demo.

Granularity

Expose primitives; keep domain logic in the agent loop. That is what lets any agent chain actions together.

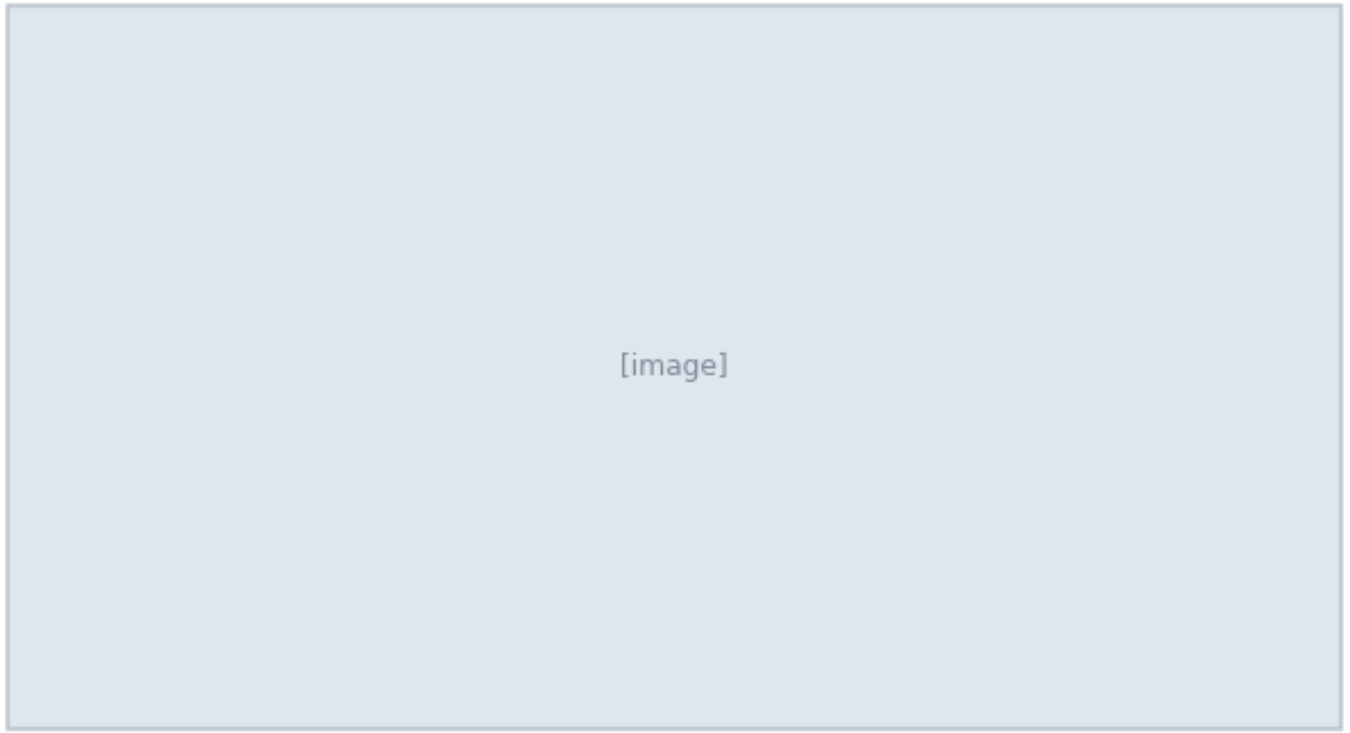
CRUD completeness

Every entity needs create, read, update, and delete. If delete is missing, the agent cannot safely clean up its own work.

Tool access is a product decision, not an implementation detail.

Context layer

Local files, project memory, and workspace state compound over time



Shared workspace

The agent and user work in the same files and see the same state. That shared state is what makes collaboration legible.

context.md pattern

Write accumulated knowledge into files the agent can re-read. Examples: brand rules, architecture notes, prior decisions, and approved defaults.

Improvement over time

Better memory makes the system smarter without shipping code. The product improves because the workspace remembers what good looks like.

Context is the moat. It compounds.

Anti-patterns

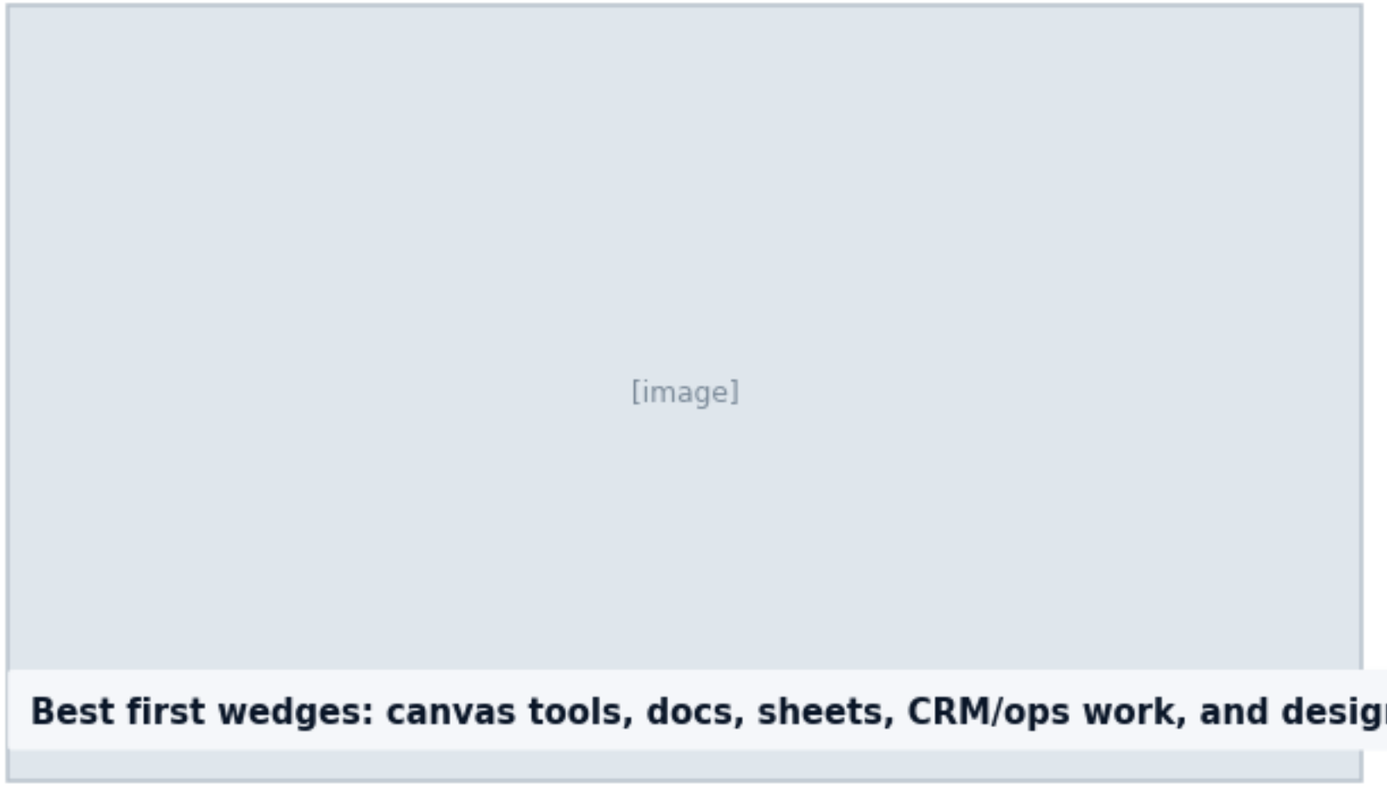
The common ways teams build something that is not actually agent-native

Agent as router	The agent only chooses which fixed function to call. That means it can route work, but it cannot reason through it.
Workflow-shaped tools	A single tool hides judgment and prevents composability. You want primitives, not a black box with a nicer prompt.
Sandbox split-brain	Agent output goes to a separate workspace the user doesn't see. If the user cannot verify state, trust breaks down fast.
Silent actions	State changes do not show up in the UI immediately. Visible feedback is how humans know the agent did the right thing.
Heuristic completion	The system guesses the task is done instead of signaling it. Completion needs explicit status, not vibes.

If the code already solved the interesting part, the agent is just a caller.

Where the opportunity is

Surfaces that already carry context are best positioned



Best first wedges: canvas tools, docs, sheets, CRM/ops work, and design workflows.

Miro / canvas tools

Shared boards can hold markdown, diagrams, and prototypes.

Notion / docs / Sheets

Knowledge bases and structured tables are easy to connect.

Adobe-class suites

Large incumbents still have room for agent-native workflows. They already own expensive, repeatable work where better context matters.

The surface is the product. The agent only makes it useful.

Why now

Three shifts make agent-native software viable today

User agents are real

People are already working inside Cursor, Codex, and similar hosts. OpenAI's Codex CLI can read, change, and run code locally.

Apps expose tools

MCP, CLI, API, and browser access make external action practical. Notion MCP can read and write workspace content with permissions.

Context is portable

Files, docs, assets, and memory can live outside a single product. Codex's in-app browser gives a shared view of rendered pages.

The category opens when the agent can bring its own context into an app, not just consume the ap

Evidence: OpenAI Codex CLI, OpenAI Codex app browser, and Notion MCP show the workflow is already here.

The opportunity is a workspace designed for the user's agent.

How to build

Five steps to find and ship an agent-native app

1	**1. Pick the job**	Define the outcome the user and agent achieve together.
2	**2. Define the surface**	Choose the canvas or workspace both can work on.
3	**3. Build for BYO agent**	Make the app useful inside Cursor/Codex-style hosts.
4	**4. Start from expertise**	Choose a domain you already know deeply.
5	**5. Plan tools early**	Scope MCP / CLI / API access on day one.

The product changes when you start with the shared surface and the tool surface together.

Step 1. Pick the job

Start with one outcome the user and agent should achieve together

Job to be done

A good wedge is narrow enough to evaluate and broad enough to repeat. Examples: • design a page • update a CRM record • reconcile a sheet

Success criteria

The user can see the result, approve it, and correct it quickly.

What the agent needs

Inputs, permissions, and a visible surface that matches the task.

Failure mode

If the agent cannot complete the work end-to-end, the job is too large.

Decision rule

Choose a job that already lives in a tool-rich workflow. The best jobs are repeated, visible, and easy to verify.

Do not start with a model feature. Start with a repeated job.

Step 2. Define the surface

Pick the shared workspace both sides can edit

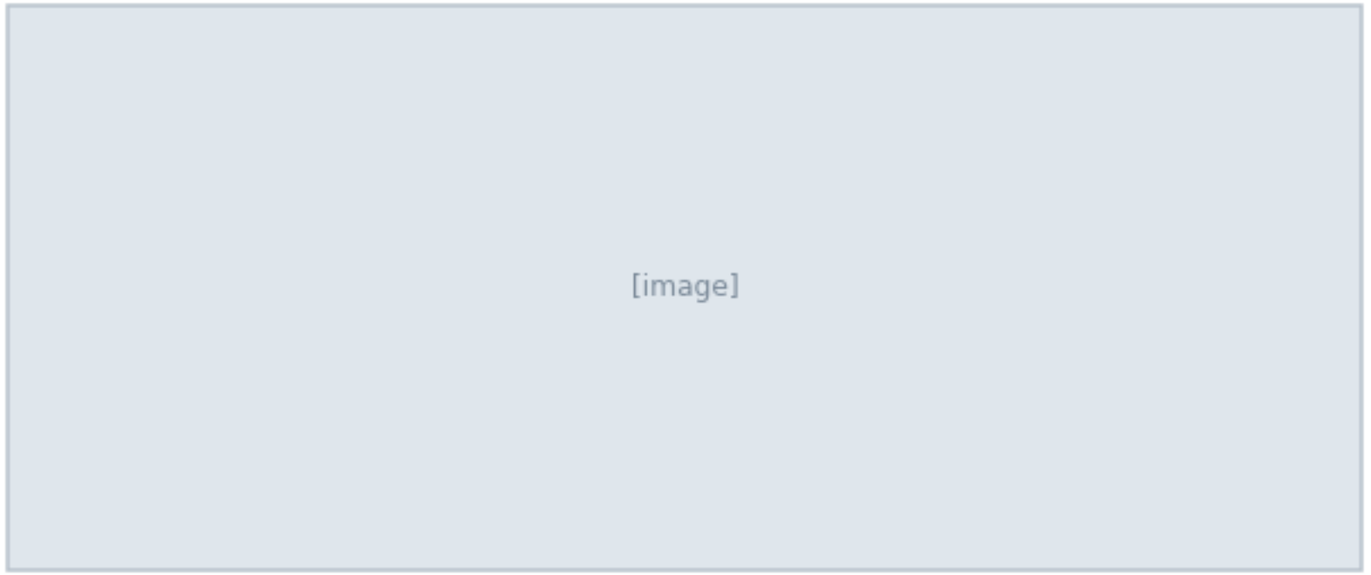
Canvas	Doc	Sheet	Browser
Use when the task is visual or spatial.	Use when the task is narrative or collaborative.	Use when the task is structured and tabular.	Use when the task needs live app interaction.

The shared surface is where the user watches the agent work, not where the model hides the work

The surface should make the collaboration obvious.

Step 3. Build for BYO agent

Design for external agents first, not an embedded chatbot



No lock-in

Do not depend on a proprietary agent living inside the app.

Tool first

Expose actions through MCP, CLI, API, and browser flows.

Shared context

Let the agent inherit local files, notes, and memory.

The app should be the place the agent works, not the place it gets trapped.

Step 4. Start from expertise

Use a domain you understand deeply enough to judge output quality

Your advantage

You know the workflow, the failure cases, the language, and the acceptable tradeoffs.

Good wedges

Product design Operations Marketing Finance Support

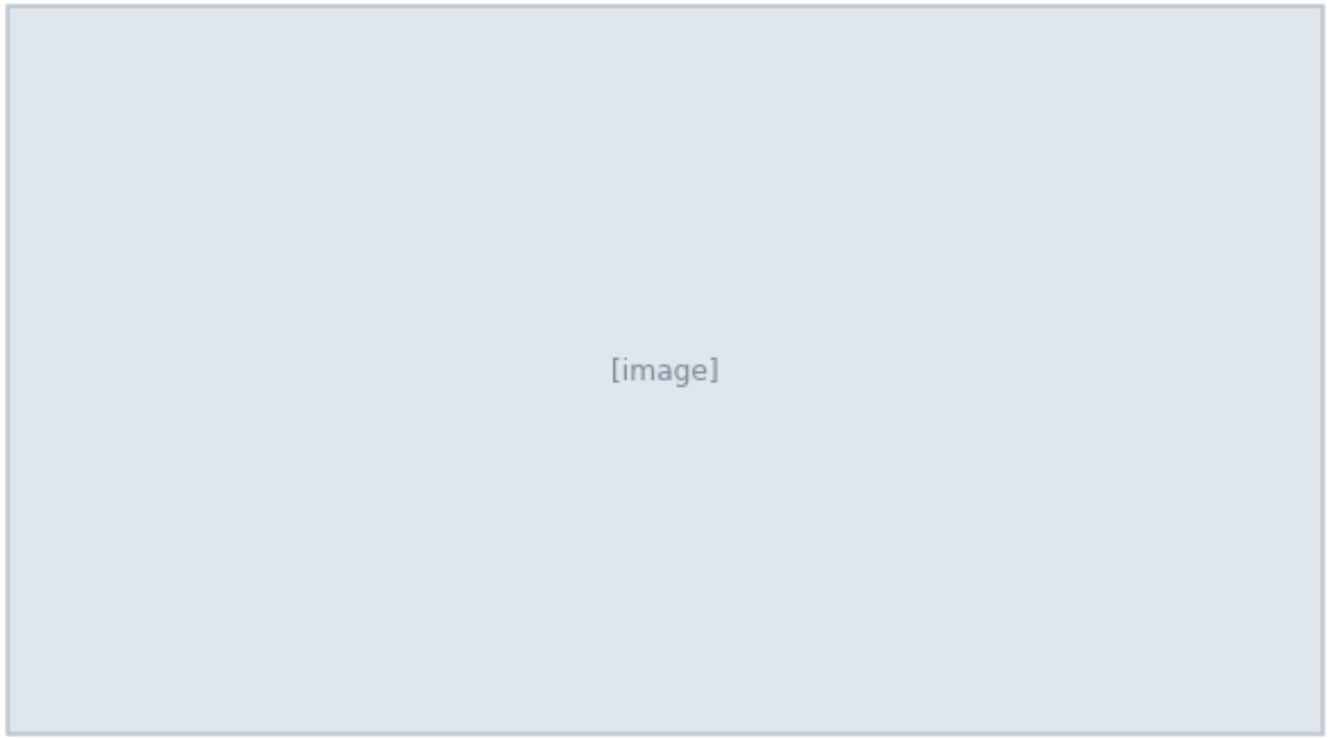
What to avoid

A generic productivity app with no domain truth. If you cannot tell good from bad output, the feedback loop will be weak.

Expertise is what lets you define the right shape for the agent.

Step 5. Plan tools early

Scope MCP / CLI / API before you build the UI around them



MCP

Best when the product needs structured tool access across clients.

CLI

Best when local agents need a simple command-line control path.

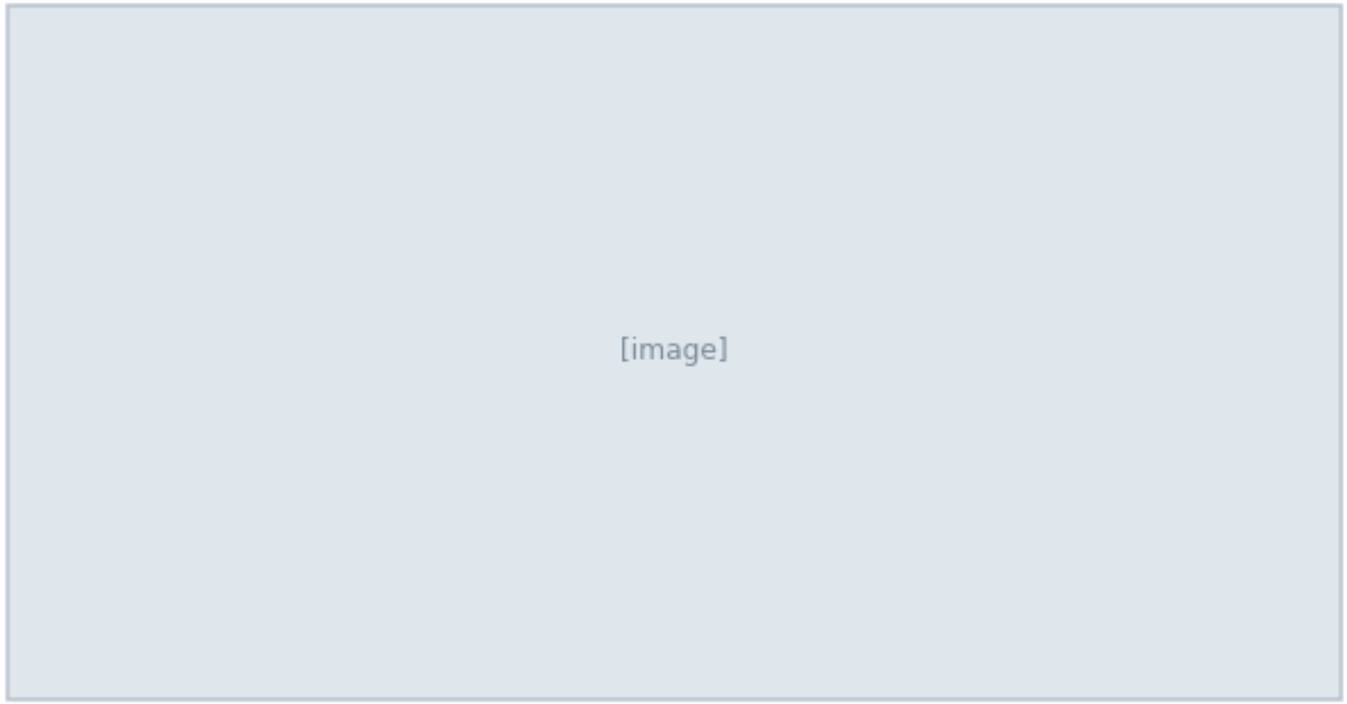
API / browser

Best when the agent must operate inside the same surface as the human.

If you cannot draw the tool map on day one, the product is still under-scoped.

Human-in-the-loop

The collaboration loop is part of the product



Watch

The user sees what the agent is doing in real time.

Correct

The user can steer the outcome before the work is locked in.

Compound

Corrections become context the agent can reuse on the next run.

Visible feedback is what turns automation into collaboration.

Examples worth studying

The strongest candidates already have shared context and shared surfaces

Miro

Canvas-first workspaces can hold diagrams, markdown, and prototypes.

Notion

Docs plus MCP make it easy to move between knowledge and action.

Sheets / Adobe

Structured tables and design suites already have rich user workflows.

The best opportunities sit where the surface already has meaning.

Launch checklist

Use this before you commit to the wedge

Can the user work here today?

If not, you are asking for behavior change first.

Can the agent reach the tools?

There must be a real tool path, not a prompt illusion.

Can both see the same state?

Shared surface means shared visibility.

Can the system improve with use?

If context cannot compound, the product will stall.

If any answer is no, the idea needs more architecture before it needs more code.

Go / no-go filter

The founder checklist for this category

User already works here?

If no one works in this surface today, you may be creating behavior from scratch.

Context exists outside the app?

Agent-native wins when the real project context is broader than one database.

Can the agent reach tools?

MCP, CLI, API, or browser access should be explicit.

Can humans watch/correct?

The visible feedback loop must be part of the product.

If you cannot answer yes to all four, the wedge is probably not ready.

Bottom line

The best new AI opportunity is software redesigned for the user's agent

Build for the user's agent, not just the user.

Design a shared surface, expose the tools, and let the agent improve with context.

The opportunity is not another chatbot. It is software that becomes useful to an external agent.

Choose a domain you know

Map the shared surface

Expose the tools early

Agent-native apps are the next wedge because they compound context, not just clicks.